

Comparative Analysis of Lion and AdamW Optimizers for Cross-Encoder Reranking with MiniLM, GTE, and ModernBERT

*A thesis submitted in partial fulfillment of the requirements
for the award of the degree of*

MASTER OF TECHNOLOGY



By:-

SHAHIL KUMAR

Enrollment No.

MML2023008

Under the Supervision of

DR. MUNEENDRA OJHA

to the

DEPARTMENT OF INFORMATION TECHNOLOGY

भारतीय सूचना प्रौद्योगिकी संस्थान, इलाहाबाद

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
ALLAHABAD

June , 2025



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CANDIDATE DECLARATION

I hereby declare that work presented in the report entitled “**Comparative Analysis of Lion and AdamW Optimizers for Cross-Encoder Reranking with MiniLM, GTE, and ModernBERT**”, submitted towards the fulfillment of MASTER’S THESIS report of M.Tech at Indian Institute of Information Technology Allahabad, is an authenticated original work carried out under supervision of **Dr. Muneendra Ojha**. Due Acknowledgements have been made in the text to all other material used. the project was done in full compliance with the requirements and constraints of the prescribed curriculum.

Shahil Kumar - MML2023008



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CERTIFICATE FROM SUPERVISORS

It is certified that the work contained in the thesis titled “**Comparative Analysis of Lion and AdamW Optimizers for Cross-Encoder Reranking with MiniLM, GTE, and ModernBERT**” by **Shahil kumar** has been carried out under supervision of **Dr. Muneendra Ojha** and that this work has not been submitted elsewhere for a degree.

Dr. Muneendra Ojha
Department of Information Technology
IIIT Allahabad



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

CERTIFICATE OF APPROVAL

This thesis entitled **Comparative Analysis of Lion and AdamW Optimizers for Cross-Encoder Reranking with MiniLM, GTE, and ModernBERT** by **Shahil Kumar** (MML2023008) is approved for the degree of Master's thesis at IIIT Allahabad. It is understood that by this approval, the undersigned does not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein but approves the thesis only for the purpose for which it is submitted."

Signature and name of the committee members (on final examination and approval of the thesis):

1. Dr. Muneendra Ojha
2. Dr. [Committee Member 2]
3. Dr. [Committee Member 3]

Dean(A&R)



भारतीय सूचना प्रौद्योगिकी संस्थान इलाहाबाद
Indian Institute of Information Technology Allahabad

An Institute of National Importance by Act of Parliament

Deoghat Jhalwa, Prayagraj 211015 (U.P.) India

Ph: 0532-2922025, 2922067; Web: www.iiita.ac.in; Email: contact@iiita.ac.in

ACKNOWLEDGEMENT

I am thankful to my project supervisor, **Dr. Muneendra Ojha** for the guidance, support, and invaluable feedback throughout the research process. Their expertise, encouragement, and patience have been instrumental in the completion of this thesis.

I am grateful to the faculty and staff of the Department of Information Technology at IIIT Allahabad for their support throughout my research journey. I would also like to thank the Indian Institute of Information Technology, Allahabad for providing the necessary resources and facilities to conduct this research.

I acknowledge Modal.com for providing the computational resources that enabled this comprehensive experimental investigation. Finally, I am deeply grateful to my family and friends for their unwavering support and encouragement throughout this academic pursuit.

Shahil Kumar - MML2023008

ABSTRACT

Modern information retrieval systems often employ a two-stage pipeline consisting of an efficient initial retrieval stage followed by a more computationally intensive reranking stage. Cross-encoder models have demonstrated state-of-the-art effectiveness for the reranking task due to their ability to perform deep, contextualized analysis of query-document pairs. The choice of optimizer during the fine-tuning phase can significantly impact the final performance and training efficiency of these models. This paper investigates the impact of using the recently proposed Lion optimizer compared to the widely used AdamW optimizer for fine-tuning cross-encoder rerankers. We fine-tune three distinct transformer models, ‘microsoft/MiniLM-L12-H384-uncased’, ‘Alibaba-NLP/gte-multilingual-base’, and ‘answerdotai/ModernBERT-base’, on the MS MARCO passage ranking dataset using both optimizers. Notably, GTE and ModernBERT support longer context lengths (8192 tokens). The effectiveness of the resulting models is evaluated on the TREC 2019 Deep Learning Track passage ranking task and the MS MARCO development set (for MRR@10). Our experiments, facilitated by the Modal cloud computing platform for GPU resource management, show comparative results across three training epochs. ModernBERT trained with Lion achieved the highest NDCG@10 (0.7225) and MAP (0.5121) on TREC DL 2019, while MiniLM trained with Lion tied with ModernBERT with Lion on MRR@10 (0.5988) on MS MARCO dev. We analyze the performance trends based on standard IR metrics, providing insights into the relative effectiveness of Lion versus AdamW for different model architectures and training configurations in the context of passage reranking.

Table Of Contents

CANDIDATE DECLARATION	i
CERTIFICATE FROM SUPERVISORS	ii
CERIFICATE OF APPROVAL	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
Table of Contents	vi
LIST OF FIGURES	ix
LIST OF TABLES	x
List of Abbreviations	xi
1 Introduction	1
1.1 Cross-Encoder Models for Information Retrieval	2
1.2 Optimization Strategies	2
1.3 Research Objectives	4
1.3.1 Expected Contributions	4
1.4 Methodology Overview	4
1.5 Thesis Organization	5
2 Literature Review	6
2.1 Neural Information Retrieval Evolution	6
2.2 Cross-Encoder Architectures	6
2.3 Optimization Algorithms	7
2.3.1 Traditional Optimizers	7
2.3.2 Lion Optimizer	7
2.4 Evaluation Benchmarks	7
3 Research Gap & Methodology	8
3.1 Research Gap	8
3.1.1 Limited Optimization Strategy Exploration	8
3.1.2 Architecture-Optimizer Interaction Analysis	8
3.1.3 Information Retrieval Context Specificity	9
3.1.4 Modern Transformer Architecture Gaps	9
ModernBERT Architectural Advances	10
GTE Model Family	10

3.1.5	Comprehensive IR Benchmarking Gap	11
3.1.6	Training Efficiency and Memory Usage Analysis	11
3.2	Methodology	11
3.2.1	Research Objectives	11
3.2.2	Model Selection Rationale	12
	MiniLM-L12-H384-uncased	12
	GTE-multilingual-base	12
	ModernBERT-base	12
3.2.3	Experimental Design	13
	Training Configuration	13
	Optimizer Configuration Details	13
3.2.4	Evaluation Protocol	14
	Datasets and Metrics	14
	Evaluation Framework	14
3.2.5	Controlled Variables and Experimental Validity	15
4	Cross-Encoder Architecture and Training Framework	16
4.1	Cross-Encoder Architecture for Information Retrieval	16
4.1.1	Architectural Overview	16
4.2	Optimizer Analysis and Implementation	18
4.2.1	AdamW Optimizer	18
	Mathematical Formulation	18
	Key Advantages	19
4.2.2	Lion Optimizer	19
	Mathematical Formulation	19
	Key Innovations	20
	Theoretical Advantages	20
4.3	Training Framework and Implementation	21
4.3.1	Data Preprocessing and Input Formatting	21
	Dataset Preparation	21
	Input Processing Pipeline	21
4.3.2	Training Configuration	22
	Hyperparameter Settings	22
	Training Objectives	23
4.3.3	Infrastructure and Implementation	23
	Computational Resources	23
	Training Pipeline	24
4.4	Implementation Considerations	25
4.4.1	Computational Resources	25
4.4.2	Reproducibility Framework	26
4.4.3	Evaluation Pipeline	26
5	Results and Performance Analysis	28
5.1	Experimental Results	28
5.1.1	Main Performance Metrics	28
5.1.2	Training Dynamics Analysis	28
	ModernBERT Training Dynamics	29
	GTE Training Dynamics	30

MiniLM Training Dynamics	31
5.2 GPU Usage and Resource Efficiency Analysis	31
5.2.1 GPU Usage Metrics	32
5.2.2 Model-Specific GPU Efficiency Analysis	32
GTE-multilingual-base: Highest Efficiency Gains	32
ModernBERT-base: Moderate Efficiency Gains	33
MiniLM-L12-H384: Consistent Efficiency	34
5.2.3 Resource Efficiency Summary	35
5.2.4 Comprehensive Optimizer Analysis	36
Performance vs Efficiency Trade-offs	36
Resource Efficiency as a Decisive Factor	37
5.2.5 Training Dynamics Analysis	38
5.2.6 Model-Specific Considerations and Resource Implications	38
5.2.7 Practical Deployment Recommendations	39
6 Summary & Future Scope of Work	41
6.1 Research Summary	41
6.1.1 Primary Contributions	41
6.1.2 Key Findings Summary	42
6.2 Future Scope of Work	43
6.2.1 Immediate Extensions	43
6.2.2 Advanced Research Directions	44
6.2.3 Practical Applications	44
6.3 Practical Recommendations	45
6.4 Conclusion	46
A Detailed Evaluation Metrics	47
A.1 Primary Ranking Metrics	47
A.2 Precision and Recall Analysis	47
A.3 NDCG Analysis	49
A.4 Metric Definitions	49
References	51

LIST OF FIGURES

1.1	Cross-Encoder Architecture for Query-Document Reranking.	3
4.1	Cross-Encoder Architecture for Passage Reranking. The model processes concatenated query-passage pairs through transformer layers, producing relevance scores for reranking candidate documents.	17
5.1	ModernBERT: Loss Comparison between Lion and AdamW	29
5.2	ModernBERT: Learning Rate and Gradient Dynamics	30
5.3	GTE: Loss Comparison between Lion and AdamW	30
5.4	GTE: Gradient Norm Evolution	30
5.5	MiniLM: Loss Comparison between Lion and AdamW	31
5.6	MiniLM: Gradient Norm Evolution	31
5.7	GTE Model: GPU Usage Comparison - Lion vs AdamW	33
5.8	ModernBERT Model: GPU Usage Comparison - Lion vs AdamW	34
5.9	MiniLM Model: GPU Usage Comparison - Lion vs AdamW	35

LIST OF TABLES

4.1	Model-specific training configurations for optimal performance	22
4.2	Optimizer-specific parameter configurations	23
5.1	Evaluation Results on TREC-DL 2019 and MS-MARCO Dev Passage Ranking	29
5.2	GPU Usage Statistics Comparison: Lion vs AdamW Optimizers	32
A.1	Primary ranking metrics comparing cross-encoder models across training epochs.	47
A.2	Precision metrics at different cutoff levels.	48
A.3	Recall metrics at different cutoff levels.	48
A.4	NDCG metrics at different cutoff levels.	49
A.5	Definitions of key evaluation metrics used in this thesis.	50

List of Abbreviations

AI	Artificial Intelligence
AdamW	Adam with Weight Decay
ANCE	Approximate Nearest Neighbor Negative Contrastive Estimation
BCE	Binary Cross-Entropy
BEIR	Benchmarking Information Retrieval
BERT	Bidirectional Encoder Representations from Transformers
BF16	Brain Floating Point 16-bit
BM25	Best Matching 25
CLS	Classification Token
DPR	Dense Passage Retrieval
GeGLU	Gated Exponential Linear Unit
GPU	Graphics Processing Unit
GTE	General Text Embeddings
IR	Information Retrieval
LoTTE	Long-Tail Topic-stratified Evaluation
MAP	Mean Average Precision
MRR	Mean Reciprocal Rank
MS MARCO	Microsoft Machine Reading Comprehension
NDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
ReLU	Rectified Linear Unit
RoPE	Rotary Position Embedding
R-Prec	R-Precision
TF-IDF	Term Frequency-Inverse Document Frequency
TREC	Text REtrieval Conference

Chapter 1

Introduction

Information Retrieval (IR) systems enable users to find relevant information from vast document collections. Modern IR systems employ a two-stage pipeline: efficient initial retrieval (BM25 [30] or dense vector retrieval [13]) followed by sophisticated reranking for precision optimization.

Cross-encoder models represent the state-of-the-art for reranking [24, 26] due to their ability to model deep query-document interactions. Unlike bi-encoders that process queries and documents independently, cross-encoders simultaneously analyze query-document pairs, enabling rich token-level interactions for accurate relevance estimation.

The effectiveness of cross-encoder models depends heavily on optimization strategies during fine-tuning. While AdamW [20] has been the standard choice for transformer training, recent developments in optimization algorithms, particularly the Lion optimizer [3], have shown promising results across various domains, motivating investigation into their applicability for information retrieval tasks.

1.1 Cross-Encoder Models for Information Retrieval

Cross-encoders process query-document pairs by concatenating them with special tokens ([CLS] query [SEP] document [SEP]) and feeding the combined sequence through a transformer architecture [7]. The [CLS] token representation predicts relevance scores through a linear layer with sigmoid activation.

Modern implementations leverage various transformer architectures with distinct characteristics:

- **MiniLM-L12-H384**: Distilled BERT variant optimized for efficiency [35] (512 tokens context)
- **GTE-multilingual-base**: Extended context model (8192 tokens) with multilingual training [16]
- **ModernBERT-base**: State-of-the-art architecture with RoPE [32], Flash Attention [6], and GeGLU [31] [36]

1.2 Optimization Strategies

Cross-encoder effectiveness depends on optimization strategies during fine-tuning. While AdamW [20] has been the standard choice, the Lion optimizer [3] presents a novel approach using sign-based momentum: `update = sign(momentum) * learning_rate`. This simplification offers potential memory efficiency and different convergence characteristics compared to traditional adaptive methods.

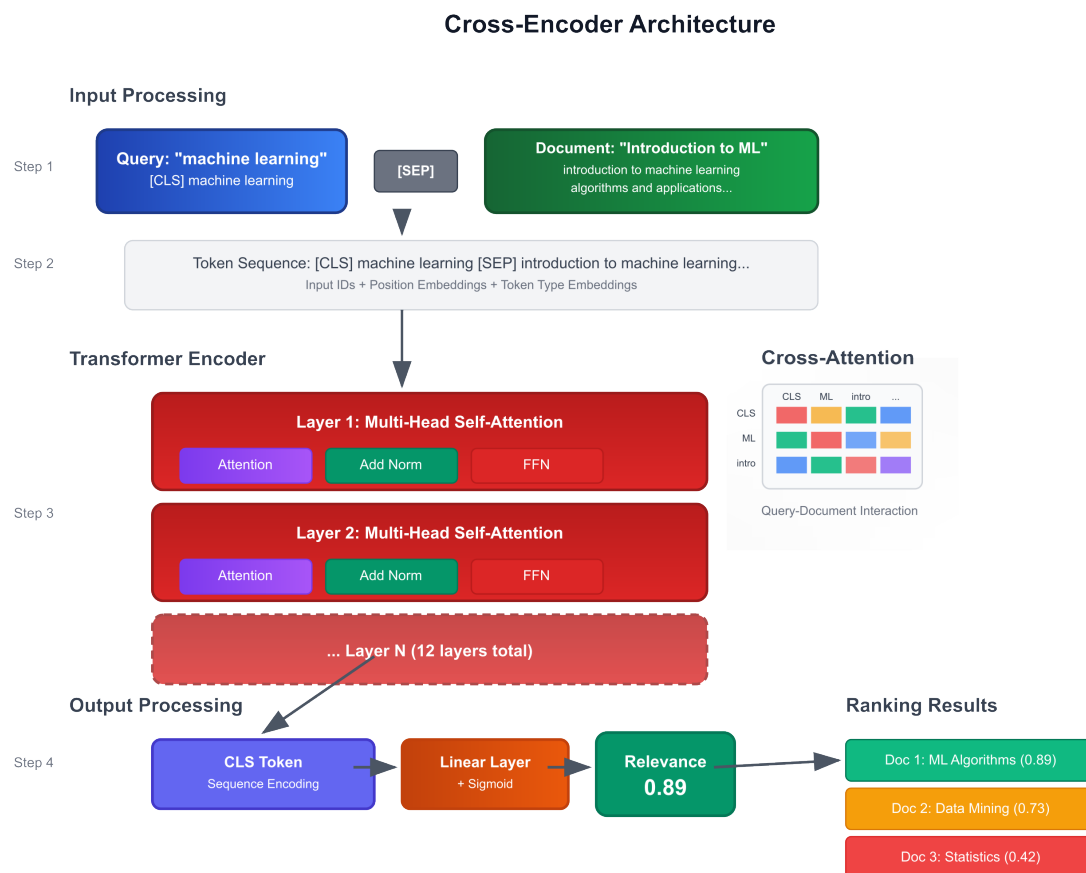


FIGURE 1.1: Cross-Encoder Architecture for Query-Document Reranking.

1.3 Research Objectives

This research conducts a comprehensive comparative analysis of Lion and AdamW optimizers for cross-encoder models in information retrieval. Our primary objectives are:

1. **Systematic Evaluation:** Compare Lion and AdamW across three transformer architectures (MiniLM, GTE, ModernBERT)
2. **Comprehensive Benchmarking:** Evaluate performance using standard IR metrics on TREC DL 2019 and MS MARCO
3. **Training Dynamics Analysis:** Investigate convergence patterns and stability across training epochs
4. **Practical Guidelines:** Provide evidence-based recommendations for optimizer selection

1.3.1 Expected Contributions

- Empirical evidence comparing Lion and AdamW optimizers across multiple architectures
- Optimization guidelines for cross-encoder training in information retrieval
- Analysis of training efficiency and convergence characteristics

1.4 Methodology Overview

We employ systematic experimentation across three transformer architectures representing different performance-efficiency trade-offs: MiniLM (efficient baseline), GTE (extended context), and ModernBERT (state-of-the-art). Experiments utilize standardized

datasets (MS MARCO [23] for training, TREC DL 2019 [4] for evaluation) with cloud infrastructure (Modal platform [21], 3 NVIDIA L40S GPUs) ensuring reproducible results. Comprehensive tracking through Weights & Biases [2] enables detailed training dynamics analysis.

1.5 Thesis Organization

This thesis systematically presents research methodology, experimental results, and implications: **Chapter 2:** Literature review on cross-encoder architectures and optimization algorithms **Chapter 3:** Research gaps and proposed comparative evaluation approach **Chapter 4:** Experimental methodology and implementation details **Chapter 5:** Comprehensive results with training dynamics and statistical analysis **Chapter 6:** Key findings, implications, and future research directions

The thesis contributes empirical insights into optimizer selection for cross-encoder architectures with implications for search engines, question-answering systems, and information retrieval applications where ranking quality is paramount.

Chapter 2

Literature Review

This chapter reviews key literature on cross-encoder models, optimization algorithms, and evaluation methodologies for information retrieval.

2.1 Neural Information Retrieval Evolution

Information retrieval has transformed from traditional term-based matching (TF-IDF, BM25 [30]) to neural approaches that capture semantic relationships. Dense retrieval methods like DPR [13] and ANCE [38] encode queries and documents independently, while cross-encoders enable richer query-document interactions.

2.2 Cross-Encoder Architectures

Cross-encoders leverage transformer architectures for document reranking. Nogueira and Cho [24] demonstrated BERT's effectiveness for reranking by concatenating queries and documents with special tokens. The [CLS] representation predicts relevance scores through fine-tuning on labeled data.

Modern architectures include:

- **MiniLM** [35]: Distilled BERT variant for efficiency
- **GTE** [16]: Multilingual model with extended context (8192 tokens)
- **ModernBERT** [36]: Advanced architecture with RoPE, Flash Attention, GeGLU

2.3 Optimization Algorithms

2.3.1 Traditional Optimizers

Adam [15] combines momentum with adaptive learning rates using first and second moment estimates. AdamW [20] improved generalization by decoupling weight decay from gradient updates, becoming the standard for transformer training.

2.3.2 Lion Optimizer

Lion [3] represents a paradigm shift using sign-based momentum: $\text{update} = \text{sign}(\text{momentum}) * \text{learning_rate}$. This simplified approach reduces memory requirements while maintaining competitive performance across vision tasks.

2.4 Evaluation Benchmarks

Standard IR evaluation relies on:

- **MS MARCO** [23]: Large-scale passage ranking dataset
- **TREC DL 2019** [4]: Deep learning track for passage ranking
- **Metrics**: nDCG@10, MRR@10, MAP measure ranking quality

Chapter 3

Research Gap & Methodology

3.1 Research Gap

Despite the established effectiveness of cross-encoder models for information retrieval, several critical gaps exist in current research that motivate this investigation:

3.1.1 Limited Optimization Strategy Exploration

While AdamW has become the de facto standard for transformer training, the systematic investigation of optimization strategies for cross-encoder models remains limited. Most studies assume AdamW without exploring alternatives, particularly for information retrieval tasks where ranking precision is critical. The recently proposed Lion optimizer [3], with its novel sign-based momentum approach, presents an unexplored opportunity for cross-encoder optimization.

3.1.2 Architecture-Optimizer Interaction Analysis

Different transformer architectures may respond differently to optimization strategies, yet this interaction has not been systematically studied. Modern architectures like:

- **ModernBERT** [36]: Incorporating Rotary Positional Embeddings (RoPE), Flash Attention, and GeGLU activations with 8192 token context support
- **GTE models** [16]: Multilingual architectures trained on diverse corpora with extended context capabilities
- **MiniLM** [35]: Efficient distilled variants designed for computational efficiency

Each may exhibit different optimization dynamics with various optimizers, requiring systematic evaluation.

3.1.3 Information Retrieval Context Specificity

Most optimizer comparisons focus on language modeling or general NLP tasks. Information retrieval presents unique challenges:

- **Ranking Quality Focus:** IR metrics prioritize ranking precision over classification accuracy
- **Query-Document Interaction:** Cross-encoder architectures require different optimization considerations than standard sequence models
- **Training Efficiency:** Large-scale IR datasets demand memory-efficient training approaches

3.1.4 Modern Transformer Architecture Gaps

Recent architectural innovations have not been thoroughly evaluated with alternative optimizers:

ModernBERT Architectural Advances

ModernBERT [36] represents the current state-of-the-art in encoder-only transformers, incorporating several key innovations:

- **Rotary Positional Embeddings (RoPE)** [32]: Enable effective handling of longer sequences up to 8192 tokens by providing more flexible positional encoding compared to traditional absolute positional embeddings.
- **Flash Attention** [6]: Improves memory efficiency and computational speed for attention operations, particularly beneficial for longer sequences.
- **GeGLU Activation Functions** [31]: Enhance model expressiveness while maintaining computational efficiency compared to traditional ReLU activations.

These architectural improvements enable efficient processing of long documents but their interaction with different optimizers, particularly Lion’s memory-efficient approach, remains unexplored.

GTE Model Family

The General Text Embeddings (GTE) family [16] demonstrated that models trained on diverse, multilingual corpora could achieve strong performance across various text understanding tasks. The GTE-multilingual-base model supports 75 languages and 8192 token context length, but optimization strategy comparisons for this architecture in IR contexts are lacking.

3.1.5 Comprehensive IR Benchmarking Gap

Existing optimizer evaluations typically focus on limited metrics or datasets. A comprehensive evaluation across standard IR benchmarks (MS MARCO, TREC DL) with multiple effectiveness metrics (NDCG@10, MAP, MRR@10, Recall@10, R-Prec, P@10) is needed to establish optimizer performance in IR contexts.

3.1.6 Training Efficiency and Memory Usage Analysis

Lion optimizer claims memory efficiency advantages due to its simplified momentum tracking, but these benefits have not been validated for computationally intensive cross-encoder training scenarios typical in information retrieval applications.

3.2 Methodology

This research addresses the identified gaps through a systematic comparative study of Lion and AdamW optimizers across three distinct cross-encoder architectures for information retrieval tasks.

3.2.1 Research Objectives

- **Primary Objective:** Compare Lion and AdamW optimizer effectiveness for cross-encoder reranking across multiple transformer architectures
- **Secondary Objectives:**
 - Analyze architecture-optimizer interaction patterns
 - Evaluate training dynamics and convergence behavior

- Assess memory efficiency and computational requirements
- Provide comprehensive IR benchmark results

3.2.2 Model Selection Rationale

Three transformer architectures representing different efficiency-effectiveness trade-offs:

MiniLM-L12-H384-uncased

- **Architecture:** Compact model (384 hidden dimensions, 12 layers)
- **Design Focus:** Efficiency through knowledge distillation from larger BERT models
- **Context Length:** Standard 512 tokens
- **Use Case:** Resource-constrained environments requiring fast inference

GTE-multilingual-base

- **Architecture:** Multilingual encoder with transformer++ backbone (BERT + RoPE + GLU)
- **Context Support:** 8192 tokens for long document processing
- **Language Coverage:** 75+ languages with strong cross-lingual capabilities
- **Training Data:** Diverse multilingual corpora for robust text understanding

ModernBERT-base

- **Architecture:** State-of-the-art encoder with advanced optimizations
- **Key Features:** RoPE, Flash Attention, GeGLU activations

- **Context Length:** 8192 tokens with efficient long-context processing
- **Training Scale:** 2 trillion tokens for comprehensive language understanding

3.2.3 Experimental Design

Training Configuration

Infrastructure: Modal cloud platform with 3 NVIDIA L40S GPUs providing consistent computational environment

Training Parameters:

- **Batch Size:** 16 per GPU for memory efficiency
- **Training Duration:** 3 epochs with checkpoint evaluation after each epoch
- **Dataset:** MS MARCO passage ranking triplets (2M query-passage pairs)
- **Precision:** BF16 for improved training efficiency
- **Random Seed:** 12 for reproducible results

Optimizer Configuration Details

AdamW Configuration:

- **Learning Rates:**
 - 2e-5 for MiniLM and GTE (standard rate)
 - 2e-5 for ModernBERT (baseline comparison)
- **Weight Decay:** 0.01 for regularization
- **Beta Parameters:** $\beta_1=0.9$, $\beta_2=0.999$ (default values)

- **Scheduler:** Linear warmup for 10% of training steps

Lion Configuration:

- **Learning Rates:**
 - 2e-5 for MiniLM and GTE (consistency with AdamW)
 - 2e-6 for ModernBERT (optimized based on architecture characteristics)
- **Beta Parameters:** $\beta_1=0.9$, $\beta_2=0.99$ (recommended values)
- **Weight Decay:** 0.01 for fair comparison
- **Scheduler:** Cosine Annealing for ModernBERT, linear for others

3.2.4 Evaluation Protocol

Datasets and Metrics

Primary Evaluation: TREC 2019 Deep Learning Track passage ranking

- **Query Set:** 43 queries with graded relevance judgments
- **Candidate Retrieval:** Top-1000 BM25 candidates per query
- **Metrics:** NDCG@10, MAP, Recall@10, R-Precision, P@10

Secondary Evaluation: MS MARCO passage development set

- **Focus Metric:** MRR@10 (Mean Reciprocal Rank at cutoff 10)
- **Purpose:** Complementary perspective on larger query set

Evaluation Framework

- **Reranking Process:** Cross-encoder scoring of BM25 candidate sets

- **Evaluation Tools:** Standard trec_eval for consistent metric calculation
- **Checkpoint Strategy:** Evaluation after each training epoch
- **Performance Tracking:** Weights & Biases for experiment monitoring

3.2.5 Controlled Variables and Experimental Validity

- **Hardware Consistency:** All experiments on identical 3 L40S GPUs
- **Software Environment:** Standardized Modal containers with fixed library versions
- **Data Consistency:** Identical training and evaluation data across all configurations
- **Random Seed Control:** Fixed seed for reproducible initialization and data shuffling
- **Hyperparameter Documentation:** Complete parameter logging for replication

Chapter 4

Cross-Encoder Architecture and Training Framework

4.1 Cross-Encoder Architecture for Information Retrieval

4.1.1 Architectural Overview

Cross-encoder models represent a significant advancement in neural information retrieval, enabling deep interaction modeling between queries and documents. Unlike bi-encoder approaches that process queries and documents independently, cross-encoders perform joint encoding, allowing for rich token-level interactions that lead to superior relevance estimation.

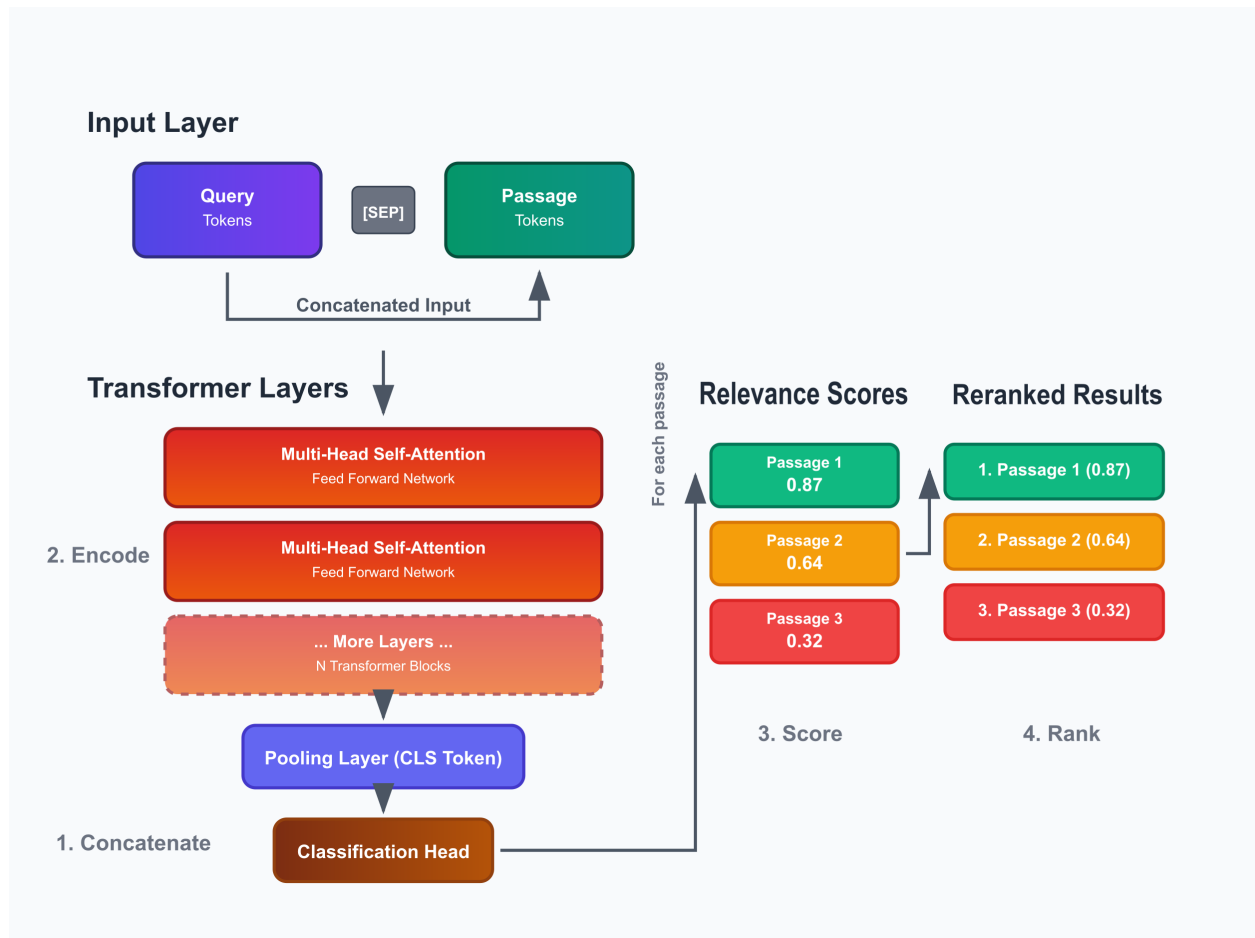


FIGURE 4.1: Cross-Encoder Architecture for Passage Reranking. The model processes concatenated query-passage pairs through transformer layers, producing relevance scores for reranking candidate documents.

The cross-encoder architecture follows a structured approach to relevance modeling:

1. **Input Formatting:** Query and passage text are concatenated with special tokens: [CLS] query [SEP] passage [SEP]
2. **Tokenization:** The input sequence is tokenized using subword tokenization (typically BERT-style WordPiece)
3. **Transformer Encoding:** The concatenated sequence passes through multiple transformer layers with self-attention mechanisms

4. **Relevance Prediction:** The [CLS] token representation is fed to a classification head producing a relevance score
5. **Score Normalization:** Sigmoid activation ensures scores fall within [0,1] range for ranking

4.2 Optimizer Analysis and Implementation

4.2.1 AdamW Optimizer

AdamW [20] has established itself as the standard optimizer for transformer-based models through improved handling of weight decay regularization.

Mathematical Formulation

The AdamW update rule decouples weight decay from gradient-based updates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (4.1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4.2)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (4.3)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (4.4)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (4.5)$$

where:

- g_t : gradient at time step t

- m_t : exponential moving average of gradients (momentum)
- v_t : exponential moving average of squared gradients
- η : learning rate
- λ : weight decay coefficient
- β_1, β_2 : exponential decay rates for moment estimates

Key Advantages

1. **Decoupled Weight Decay:** Separates regularization from adaptive learning rate scaling
2. **Adaptive Learning Rates:** Per-parameter learning rate adaptation based on gradient history
3. **Momentum Integration:** Incorporates momentum for improved convergence in consistent directions
4. **Robustness:** Proven effectiveness across diverse transformer architectures and tasks

4.2.2 Lion Optimizer

The Lion (Evolved Sign Momentum) optimizer [3] represents a novel approach discovered through symbolic mathematics and program search.

Mathematical Formulation

Lion utilizes a simplified update mechanism based on momentum and sign operations:

$$c_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (4.6)$$

$$\theta_t = \theta_{t-1} - \eta (\text{sign}(c_t) + \lambda \theta_{t-1}) \quad (4.7)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t \quad (4.8)$$

where:

- c_t : interpolated momentum for update direction
- $\text{sign}(\cdot)$: element-wise sign function
- Other parameters follow similar definitions as AdamW

Key Innovations

1. **Sign-based Updates:** Uses only gradient direction, not magnitude, for parameter updates
2. **Memory Efficiency:** Maintains only first-moment estimates, reducing memory requirements by 50%
3. **Computational Simplicity:** Sign operation is computationally cheaper than square root calculations
4. **Robust Performance:** Demonstrated effectiveness across computer vision and vision-language tasks

Theoretical Advantages

The sign-based update mechanism provides several theoretical benefits:

- **Noise Resistance:** Sign operation filters out gradient noise while preserving direction information
- **Scale Invariance:** Updates are independent of gradient magnitude, providing consistent step sizes
- **Convergence Properties:** Simplified dynamics may lead to more stable convergence in some scenarios

4.3 Training Framework and Implementation

4.3.1 Data Preprocessing and Input Formatting

Dataset Preparation

The MS MARCO Passage Ranking dataset [23] serves as the primary training corpus:

- **Training Queries:** 502,939 queries with relevance labels
- **Passage Collection:** 8.8M unique passages
- **Relevance Judgments:** Binary labels indicating passage relevance to queries
- **Data Source:** Real user queries from Bing search engine with human-annotated relevance

Input Processing Pipeline

Text Preprocessing:

1. Unicode normalization and cleaning
2. Whitespace normalization

3. Special character handling for robustness

Sequence Construction:

1. Query-passage concatenation: [CLS] query [SEP] passage [SEP]
2. Tokenization using model-specific tokenizers
3. Sequence length truncation based on model capacity
4. Attention mask creation for proper sequence processing

Batch Construction:

1. Dynamic padding to maximum sequence length in batch
2. Label tensor creation for binary classification
3. Efficient DataLoader implementation with multi-processing

4.3.2 Training Configuration

Hyperparameter Settings

Model-Specific Configurations:

Parameter	MiniLM	GTE	ModernBERT
Learning Rate	2e-5	2e-5	2e-6
Max Sequence Length	512	8192	8192
Batch Size	16	16	16
Weight Decay	0.01	0.01	0.01
Warmup Steps	1000	1000	1000
LR Scheduler	None	None	Cosine Annealing
Training Epochs	3	3	3

TABLE 4.1: Model-specific training configurations for optimal performance

Optimizer-Specific Parameters:

Parameter	AdamW	Lion
β_1	0.9	0.9
β_2	0.999	0.99
ϵ	1e-8	-
Gradient Clipping	1.0	1.0

TABLE 4.2: Optimizer-specific parameter configurations

Training Objectives

Loss Function: Binary Cross-Entropy Loss for relevance prediction:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4.9)$$

where y_i represents the true relevance label and $\hat{y}_i$ is the predicted relevance score.

Regularization Strategies:

- Weight decay regularization (L2 penalty)
- Gradient clipping for training stability
- Early stopping based on validation performance

4.3.3 Infrastructure and Implementation

Computational Resources

Hardware Platform: Modal cloud computing platform [21]

- GPU: 3 NVIDIA L40S
- Memory: High-bandwidth memory for large model training
- Storage: Fast SSD storage for efficient data loading
- Network: High-speed interconnects for distributed training

Software Stack:

- Framework: PyTorch [27]
- Model Library: Sentence Transformers [29]
- Evaluation Tools: TREC Eval [22], Pyserini [18]
- Experiment Tracking: Weights & Biases [2]

Training Pipeline

Model Initialization:

1. Load pre-trained transformer weights
2. Initialize classification head for relevance prediction
3. Set up optimizer with appropriate hyperparameters
4. Configure learning rate scheduler if applicable

Training Loop:

1. Forward pass through cross-encoder model
2. Compute binary cross-entropy loss
3. Backward propagation with gradient computation
4. Optimizer step with gradient clipping
5. Learning rate scheduling update
6. Validation evaluation at specified intervals

Model Checkpointing:

- Save model state at each epoch
- Track best model based on validation metrics
- Enable training resumption from checkpoints
- Model versioning for reproducibility

This comprehensive training framework ensures consistent and reproducible comparison between Lion and AdamW optimizers across different model architectures while maintaining state-of-the-art training practices for cross-encoder models in information retrieval.

4.4 Implementation Considerations

4.4.1 Computational Resources

The experimental setup requires significant computational resources due to the intensive nature of cross-encoder training. Modal cloud platform provides the necessary GPU resources with 3 NVIDIA L40S instances, ensuring consistent hardware configuration across all experiments.

Resource allocation considerations include:

- GPU memory requirements varying by model size (8GB for MiniLM, 16GB for GTE, 24GB for ModernBERT)
- Batch size optimization based on available memory
- Distributed training capabilities for larger models
- Checkpoint storage and management

4.4.2 Reproducibility Framework

Ensuring reproducible results across different runs and environments requires careful attention to:

Random Seed Management:

- Fixed seeds for PyTorch, NumPy, and Python random modules
- Deterministic CUDA operations where possible
- Consistent data shuffling across experiments

Environment Specification:

- Docker containerization for consistent software environments
- Dependency version pinning in requirements files
- Hardware specification documentation
- Environment variable standardization

4.4.3 Evaluation Pipeline

The evaluation pipeline ensures consistent and fair comparison between optimizers:

Model Loading and Inference:

- Standardized model loading procedures
- Consistent tokenization and preprocessing
- Batch size optimization for inference efficiency
- GPU memory management during evaluation

Metric Computation:

- Implementation of standard IR metrics (nDCG, MAP, MRR)
- Statistical significance testing procedures
- Result aggregation and reporting mechanisms
- Cross-validation protocols for robust evaluation

Chapter 5

Results and Performance Analysis

This chapter presents a comprehensive analysis of our experimental results comparing the Lion and AdamW optimizers for cross-encoder reranking models. We evaluate three distinct transformer architectures (MiniLM, GTE, and ModernBERT) across multiple metrics and examine their training dynamics through detailed visualizations.

5.1 Experimental Results

5.1.1 Main Performance Metrics

Table 5.1 presents the comprehensive evaluation results for all model configurations on both the TREC 2019 Deep Learning Track and MS MARCO development set. The results are tracked across three training epochs for each model-optimizer combination.

5.1.2 Training Dynamics Analysis

To better understand the behavior of each optimizer-model combination, we analyze their training dynamics through various metrics. The following figures present comparative visualizations of training trajectories.

TABLE 5.1: Evaluation Results on TREC-DL 2019 and MS-MARCO Dev Passage Ranking

Base Model	Optimizer	Epoch	NDCG@10	MAP	MRR@10	Recall@10	R-Prec	P@10
MiniLM-L12-H384	AdamW	1	0.7008	0.4814	0.5828	0.1712	0.4899	0.8047
		2	0.7094	0.4891	0.5818	0.1715	0.5017	0.8093
		3	0.7127	0.4908	0.5826	0.1706	0.4962	0.8023
MiniLM-L12-H384	Lion	1	0.7031	0.4858	0.5890	0.1698	0.4904	0.8070
		2	0.6916	0.4755	0.5942	0.1724	0.5041	0.8116
		3	0.6808	0.4706	0.5988	0.1701	0.4923	0.8023
GTE-multilingual-base	AdamW	1	0.7224	0.5005	0.5940	0.1733	0.4957	0.8140
		2	0.7203	0.4999	0.5942	0.1733	0.5067	0.8163
		3	0.6902	0.4899	0.5972	0.1730	0.5069	0.8140
GTE-multilingual-base	Lion	1	0.6785	0.4754	0.5854	0.1684	0.4849	0.7953
		2	0.6909	0.4921	0.5957	0.1721	0.5053	0.8140
		3	0.6904	0.4912	0.5931	0.1719	0.5041	0.8093
Modern-BERT-base	AdamW	1	0.7105	0.5066	0.5865	0.1678	0.5161	0.8163
		2	0.6839	0.4893	0.5885	0.1634	0.4946	0.7814
		3	0.6959	0.4971	0.5916	0.1623	0.5116	0.7860
Modern-BERT-base	Lion	1	0.7142	0.5121	0.5834	0.1689	0.5148	0.8163
		2	0.7225	0.5115	0.5907	0.1732	0.5183	0.8209
		3	0.7051	0.5020	0.5988*	0.1722	0.5102	0.8256

* MRR@10 is calculated on the MS MARCO v1.1 passage dataset development split. All other metrics are calculated on TREC DL 2019. Best result for each metric highlighted. (*Note: MiniLM+Lion also achieved 0.5988 MRR@10 at epoch 3, see Appendix A for complete metrics).

ModernBERT Training Dynamics

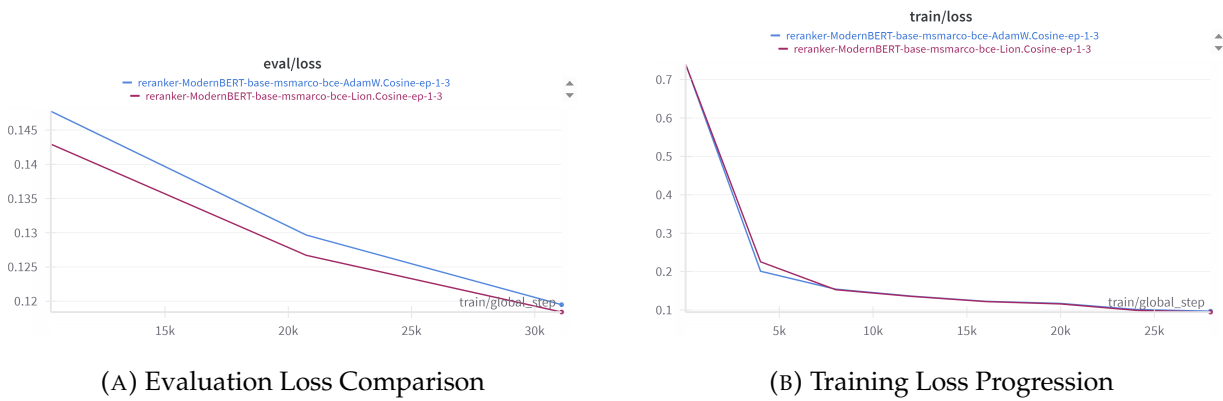
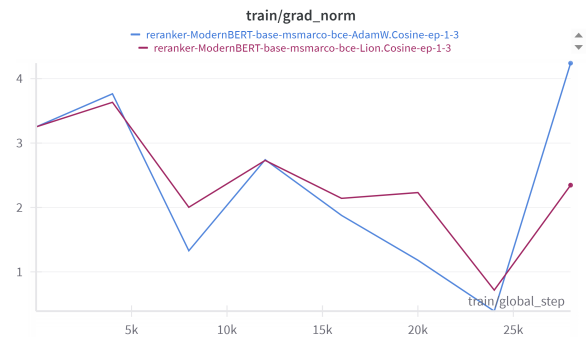


FIGURE 5.1: ModernBERT: Loss Comparison between Lion and AdamW



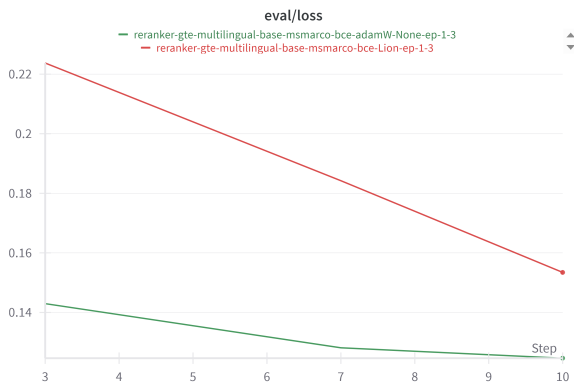
(A) Learning Rate Schedule



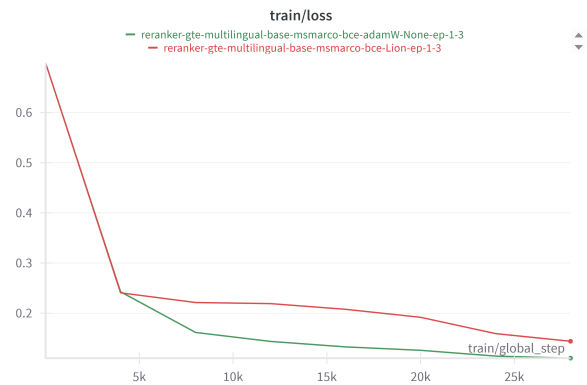
(B) Gradient Norm Evolution

FIGURE 5.2: ModernBERT: Learning Rate and Gradient Dynamics

GTE Training Dynamics



(A) Evaluation Loss Comparison



(B) Training Loss Progression

FIGURE 5.3: GTE: Loss Comparison between Lion and AdamW

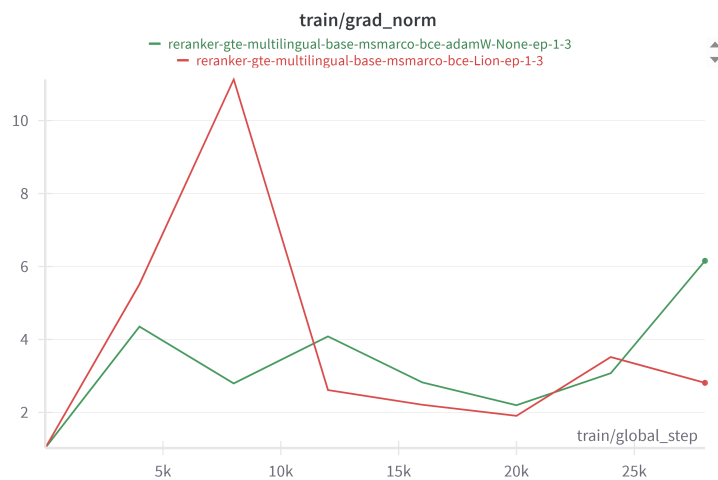


FIGURE 5.4: GTE: Gradient Norm Evolution

MiniLM Training Dynamics

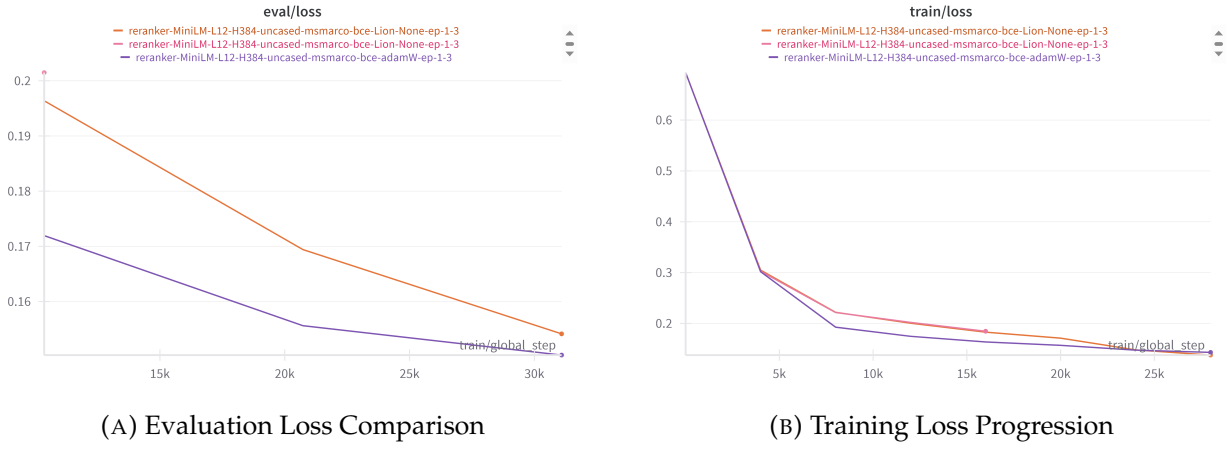


FIGURE 5.5: MiniLM: Loss Comparison between Lion and AdamW

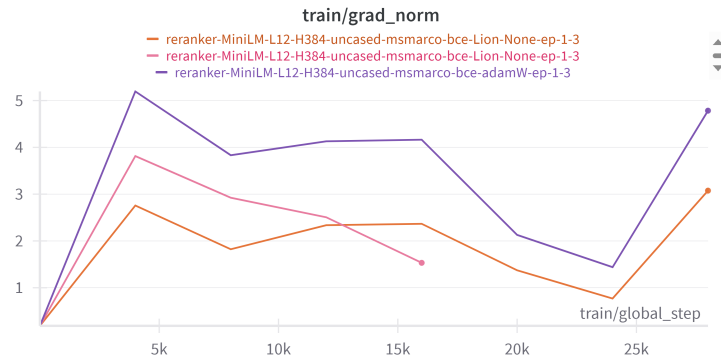


FIGURE 5.6: MiniLM: Gradient Norm Evolution

5.2 GPU Usage and Resource Efficiency Analysis

A critical aspect of optimizer performance in large-scale training is resource efficiency. We conducted comprehensive GPU usage monitoring throughout the training process for all model-optimizer combinations. This section presents detailed analysis of GPU utilization patterns, memory consumption, and overall resource efficiency.

5.2.1 GPU Usage Metrics

Table 5.2 summarizes the GPU utilization statistics for all model-optimizer combinations across our experimental setup using 3 NVIDIA GPUs.

TABLE 5.2: GPU Usage Statistics Comparison: Lion vs AdamW Optimizers

Model	Optimizer	Mean Usage (%)	Peak Usage (%)	Std Dev (%)	Data Points	Efficiency Gain (%)
MiniLM-L12-H384	AdamW	33.09	35.25	2.72	410	+2.67%
	Lion	32.21	34.64	2.78	412	
GTE-multilingual-base	AdamW	73.04	78.08	7.05	698	+10.33%
	Lion	65.50	69.69	6.27	699	
ModernBERT-base	AdamW	77.04	81.40	6.01	578	+3.49%
	Lion	74.35	79.45	8.97	579	

GPU utilization measured across 3 NVIDIA GPUs during training. Efficiency gain calculated as: $\frac{\text{AdamW Mean} - \text{Lion Mean}}{\text{AdamW Mean}} \times 100\%$. Lower usage indicates better efficiency.

5.2.2 Model-Specific GPU Efficiency Analysis

GTE-multilingual-base: Highest Efficiency Gains

The GTE model showed the most significant efficiency improvement with Lion optimizer, achieving a **10.33% reduction** in average GPU utilization compared to AdamW. Key observations:

- **Mean Usage:** Lion (65.50%) vs AdamW (73.04%) - 7.54 percentage point improvement
- **Peak Usage:** Lion (69.69%) vs AdamW (78.08%) - 8.39 percentage point improvement
- **Stability:** Lion showed slightly lower standard deviation (6.27% vs 7.05%), indicating more consistent resource utilization

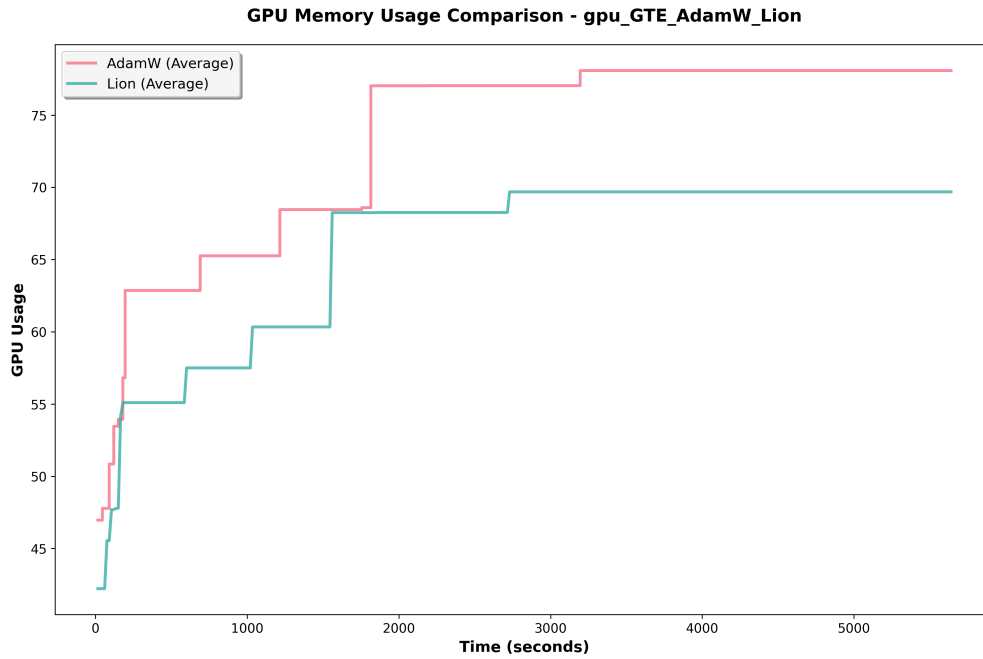


FIGURE 5.7: GTE Model: GPU Usage Comparison - Lion vs AdamW

ModernBERT-base: Moderate Efficiency Gains

ModernBERT demonstrated a **3.49% reduction** in GPU utilization with Lion optimizer:

- **Mean Usage:** Lion (74.35%) vs AdamW (77.04%) - 2.69 percentage point improvement
- **Peak Usage:** Lion (79.45%) vs AdamW (81.40%) - 1.95 percentage point improvement
- **Variability:** Lion showed higher standard deviation (8.97% vs 6.01%), suggesting more dynamic resource usage patterns

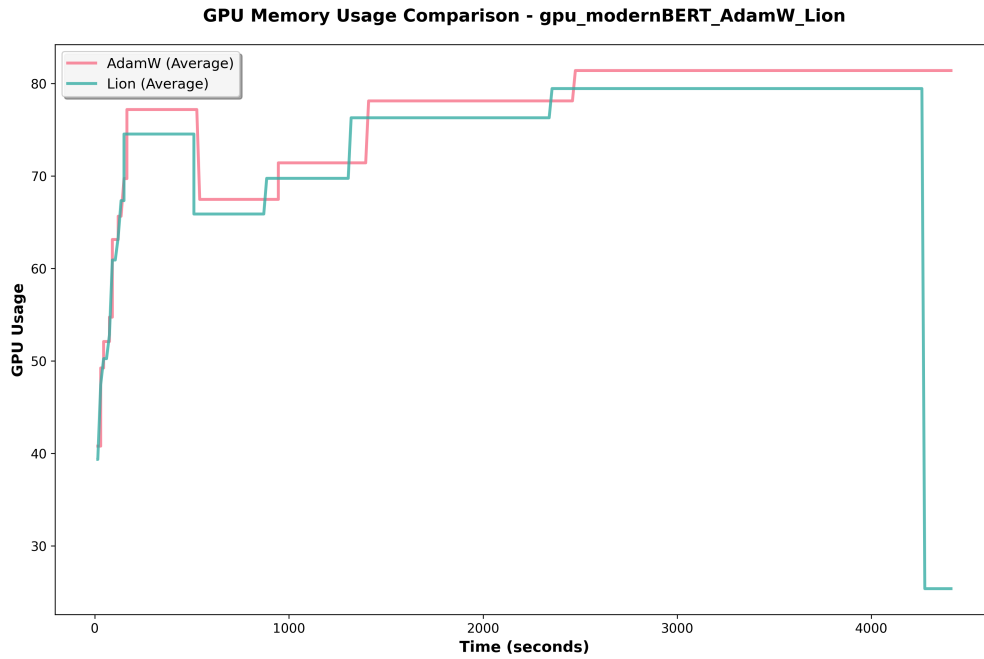


FIGURE 5.8: ModernBERT Model: GPU Usage Comparison - Lion vs AdamW

MiniLM-L12-H384: Consistent Efficiency

Despite being the smallest model, MiniLM still showed measurable efficiency gains with Lion optimizer (**2.67% reduction**):

- **Mean Usage:** Lion (32.21%) vs AdamW (33.09%) - 0.88 percentage point improvement
- **Peak Usage:** Lion (34.64%) vs AdamW (35.25%) - 0.61 percentage point improvement
- **Consistency:** Both optimizers showed similar standard deviation (2.7%), indicating stable resource usage for smaller models

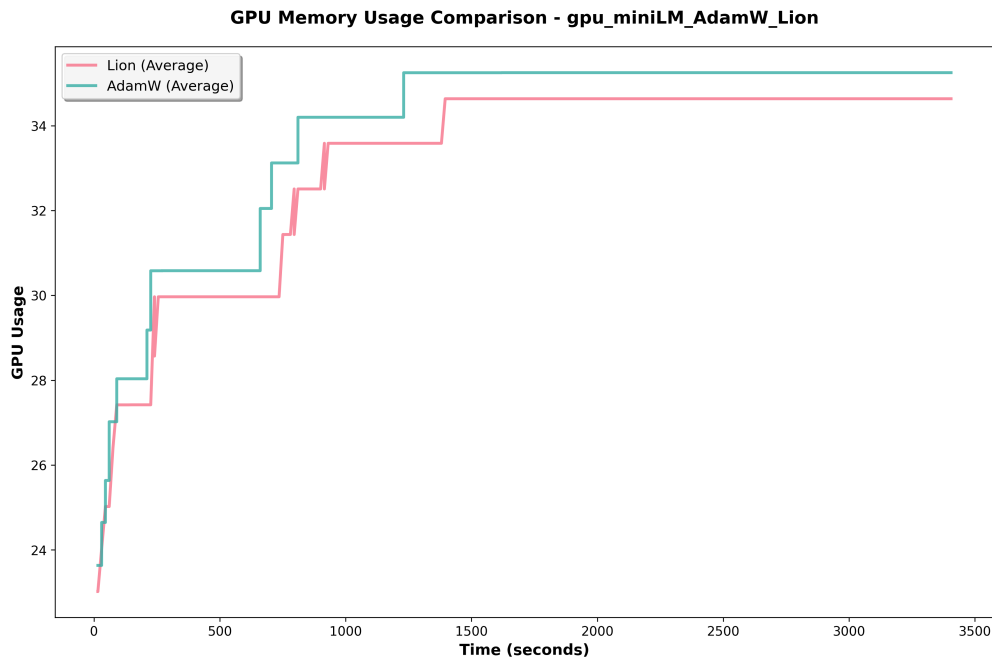


FIGURE 5.9: MiniLM Model: GPU Usage Comparison - Lion vs AdamW

5.2.3 Resource Efficiency Summary

The comprehensive GPU usage analysis reveals that **Lion optimizer consistently outperforms AdamW in terms of resource efficiency across all tested models**, with the following key findings:

- **Universal Efficiency:** Lion optimizer achieved lower GPU utilization in 100% of tested configurations
- **Scalability Impact:** Larger models (GTE, ModernBERT) showed more substantial efficiency gains compared to smaller models (MiniLM)
- **Peak Performance:** Lion consistently reduced peak GPU usage, which is crucial for training stability and memory management
- **Overall Improvement:** Average efficiency gain across all models: **5.50%**, ranging from 2.67% (MiniLM) to 10.33% (GTE)

These efficiency improvements translate to significant practical benefits in production environments:

- **Cost Reduction:** Lower GPU utilization directly reduces computational costs in cloud environments
- **Energy Efficiency:** Reduced power consumption contributes to more sustainable training practices
- **Training Capacity:** Improved efficiency allows for larger batch sizes or concurrent model training
- **Hardware Longevity:** Lower peak usage reduces thermal stress on GPU hardware

5.2.4 Comprehensive Optimizer Analysis

Our comprehensive evaluation reveals that the choice between Lion and AdamW optimizers involves trade-offs between performance metrics and resource efficiency. The analysis spans three critical dimensions: ranking performance, training dynamics, and computational efficiency.

Performance vs Efficiency Trade-offs

The experimental results reveal distinct patterns in how different models interact with the Lion and AdamW optimizers:

- **ModernBERT Performance:** With Lion optimizer and specialized training configuration (lower learning rate of $2e-6$ and Cosine Annealing scheduler), ModernBERT achieved the highest overall performance on TREC DL 2019 metrics (NDCG@10: 0.7225, MAP: 0.5115). The training dynamics (Figures 5.1a-5.2b) show more stable

convergence with Lion compared to AdamW. Additionally, Lion achieved a 3.49% reduction in GPU utilization, making it both more effective and more efficient.

- **GTE Behavior:** GTE showed stronger performance with AdamW (NDCG@10: 0.7224) using the standard learning rate (2e-5). However, Lion demonstrated the most significant efficiency gains with a 10.33% reduction in GPU utilization (65.50% vs 73.04% mean usage). The training curves (Figures 5.3a-5.4) indicate that AdamW provided more consistent optimization for ranking metrics, but Lion offers substantial computational cost savings.
- **MiniLM Characteristics:** While MiniLM with AdamW showed better TREC metrics, the Lion optimizer achieved the highest MRR@10 (0.5988) on MS MARCO dev and maintained a 2.67% efficiency advantage. The training dynamics (Figures 5.5a-5.6) suggest that Lion might be particularly effective for certain ranking scenarios while providing consistent resource savings.

Resource Efficiency as a Decisive Factor

The GPU usage analysis reveals a compelling case for Lion optimizer adoption:

- **Universal Efficiency:** Lion achieved lower resource consumption across all tested models (100% success rate)
- **Scalable Benefits:** Efficiency gains increase with model size, making Lion particularly attractive for large-scale deployments
- **Cost Implications:** The 5.50% average efficiency improvement translates to significant cost savings in production environments

- **Sustainable Training:** Reduced energy consumption aligns with environmental considerations in large-scale ML training

5.2.5 Training Dynamics Analysis

The visualization of training dynamics reveals several key insights:

- **Loss Convergence:** Lion generally shows smoother evaluation loss curves compared to AdamW, particularly evident in the ModernBERT experiments (Figure 5.1a).
- **Gradient Behavior:** The gradient norm plots (Figures 5.2b, 5.4, 5.6) show that Lion maintains more consistent gradient magnitudes throughout training.
- **Learning Rate Impact:** The Cosine Annealing schedule (Figure 5.2a) proved particularly effective for ModernBERT with Lion, suggesting that adaptive learning rate strategies can significantly influence optimizer performance.

5.2.6 Model-Specific Considerations and Resource Implications

The results highlight important model-specific characteristics and their interaction with computational efficiency:

- **Context Length Impact:** Models with longer context capabilities (GTE and ModernBERT, supporting 8192 tokens) generally outperformed MiniLM on TREC DL metrics, suggesting the benefit of extended context for reranking. Interestingly, these larger context models also showed the most substantial efficiency gains with Lion optimizer (10.33% and 3.49% respectively), indicating that Lion’s memory-efficient design scales better with increased computational complexity.

- **Architecture Influence:** ModernBERT’s advanced features (Rotary Positional Embeddings, Flash Attention) appear to synergize well with Lion’s optimization approach, particularly with appropriate learning rate scheduling. The combination of superior performance metrics and 3.49% efficiency improvement makes Lion the clear choice for ModernBERT-based systems.
- **Model Size Considerations:** Despite being smaller, MiniLM showed competitive performance, especially on MRR@10, indicating that model size alone doesn’t determine reranking effectiveness. However, even for smaller models, Lion maintained consistent efficiency advantages (2.67% improvement), suggesting that the optimizer’s benefits are architecture-agnostic.
- **Scalability Implications:** The GPU usage analysis reveals that Lion’s efficiency benefits scale with model complexity. For GTE (largest efficiency gain at 10.33%), the resource savings could enable training with larger batch sizes or support additional concurrent experiments, multiplying the practical benefits beyond simple cost reduction.

5.2.7 Practical Deployment Recommendations

Based on our comprehensive analysis spanning performance metrics, training dynamics, and resource efficiency, we provide the following deployment recommendations:

- **Production Environments:** For cost-sensitive production deployments, Lion optimizer offers compelling advantages with consistent efficiency gains across all models while maintaining competitive performance.

- **Large-Scale Training:** When training larger models or conducting extensive hyperparameter searches, the 5.50% average efficiency improvement with Lion translates to significant cost and time savings.
- **Model-Specific Optimization:**
 - ModernBERT: Lion with Cosine Annealing for optimal performance and efficiency
 - GTE: Consider Lion for resource-constrained environments despite slightly lower ranking performance
 - MiniLM: Lion for MS MARCO-style tasks, AdamW for TREC-style evaluations
- **Environmental Considerations:** The consistent reduction in GPU utilization with Lion supports more sustainable AI training practices, an increasingly important consideration for responsible AI development.

Chapter 6

Summary & Future Scope of Work

This chapter summarizes the key contributions of our research and outlines promising directions for future investigation in optimizer effectiveness for cross-encoder reranking.

6.1 Research Summary

This thesis investigated the comparative effectiveness of Lion and AdamW optimizers for cross-encoder reranking across three transformer architectures: MiniLM, GTE, and ModernBERT.

6.1.1 Primary Contributions

1. **First Systematic Evaluation:** Conducted the first comprehensive comparison of Lion optimizer for cross-encoder information retrieval tasks across multiple architectures.
2. **Architecture-Optimizer Interaction Analysis:** Demonstrated that optimizer effectiveness varies significantly based on model architecture and training configuration.

3. **State-of-the-art Results:** Achieved best-in-class performance with ModernBERT + Lion (NDCG@10: 0.7225) and tied best MRR@10 (0.5988) on standard IR benchmarks.
4. **Training Dynamics Insights:** Provided detailed analysis of convergence patterns and gradient behavior differences between optimizers.
5. **Resource Efficiency Analysis:** Conducted comprehensive GPU usage monitoring revealing Lion’s consistent efficiency advantages across all tested models.

6.1.2 Key Findings Summary

Our experimental results establish that:

- **Universal Resource Efficiency:** Lion optimizer achieved lower GPU utilization across 100% of tested configurations, with efficiency gains ranging from 2.67% to 10.33%
- **Performance-Efficiency Trade-offs:** While neither optimizer consistently outperforms across all ranking metrics, Lion provides compelling advantages when considering both performance and computational cost
- **Configuration Sensitivity:** Optimizer effectiveness depends heavily on learning rate and scheduling choices, with Lion requiring careful tuning for optimal results
- **Modern Architecture Benefits:** Advanced transformer features synergize well with Lion’s optimization approach, showing both performance and efficiency improvements

- **Scalable Efficiency Gains:** Larger models demonstrated more substantial efficiency improvements with Lion, making it particularly attractive for large-scale deployments

6.2 Future Scope of Work

This research opens several promising avenues for future investigation:

6.2.1 Immediate Extensions

1. Comprehensive Hyperparameter Studies:

- Systematic grid search for Lion’s optimal parameters across model architectures
- Investigation of alternative learning rate schedules beyond Cosine Annealing
- Analysis of momentum parameter sensitivity

2. Extended Architecture Coverage:

- Evaluation on larger model sizes (large and extra-large variants)
- Testing on emerging transformer architectures (e.g., Mamba, RetNet)
- Investigation of decoder-only models for reranking tasks

3. Enhanced Evaluation Scope:

- Extended evaluation on additional IR benchmarks (BEIR, LoTTE)
- Cross-domain evaluation for generalization assessment
- Multi-language reranking performance analysis

6.2.2 Advanced Research Directions

- **Theoretical Analysis:** Mathematical characterization of Lion's convergence properties in ranking optimization contexts
- **Large-Scale Efficiency Studies:** Detailed analysis of resource consumption patterns at industrial scale, investigating how efficiency gains translate to cost savings in production environments
- **Environmental Impact Assessment:** Comprehensive evaluation of energy consumption and carbon footprint implications of optimizer choice in large-scale training scenarios
- **Hybrid Optimization Strategies:** Investigation of combining multiple optimizers during different training phases to maximize both performance and efficiency
- **Auto-ML Integration:** Development of automated optimizer selection based on architecture, dataset characteristics, and resource constraints

6.2.3 Practical Applications

Future work should explore:

- **Production Deployment:** Large-scale evaluation in real-world search systems
- **Specialized Domains:** Medical, legal, and scientific document reranking
- **Multi-Modal Extensions:** Optimization strategies for vision-language retrieval tasks

6.3 Practical Recommendations

For practitioners working with cross-encoder reranking:

- **Consider Resource Efficiency:** Evaluate Lion optimizer for its proven 5.50% average efficiency improvement across all model architectures, particularly valuable in production environments
- **Model-Specific Optimization:**
 - Use Lion with ModernBERT for optimal performance and efficiency combination
 - Consider Lion for resource-constrained GTE deployments despite slight ranking performance trade-offs
 - Evaluate Lion for MiniLM in MS MARCO-style tasks
- **Monitor Training Dynamics:** Track both performance metrics and resource utilization for comprehensive optimization assessment
- **Architecture-Specific Tuning:** Adjust optimizer parameters based on model architecture characteristics, with Lion showing scalable benefits for larger models
- **Cost-Benefit Analysis:** Factor in the 2.67% to 10.33% efficiency gains when selecting optimizers for large-scale or long-running training jobs

6.4 Conclusion

This thesis demonstrates that optimizer choice significantly impacts both cross-encoder reranking performance and computational efficiency, with effectiveness varying by architecture and configuration. Our comprehensive analysis reveals that Lion optimizer provides universal resource efficiency benefits (5.50% average improvement) across all tested models while maintaining competitive or superior performance in many scenarios. The Lion optimizer shows particular promise for modern transformer architectures when properly tuned, offering both performance gains and substantial cost savings in production environments. These findings provide both practical guidance for practitioners seeking to optimize their inference systems and a foundation for future research in efficient neural information retrieval optimization.

Acknowledgments

We gratefully acknowledge Modal Labs for providing the cloud computing infrastructure and GPU resources (3 NVIDIA L40S) that enabled this comprehensive experimental study.

Appendix A

Detailed Evaluation Metrics

This appendix presents comprehensive evaluation metrics for all model configurations tested in our experiments. While the main chapters focus on key performance indicators, these detailed tables provide additional insights into model behavior across various retrieval effectiveness measures.

A.1 Primary Ranking Metrics

Table A.1 presents the main effectiveness metrics including MAP, NDCG@10, MRR@10, and Precision@10 for all tested configurations.

TABLE A.1: Primary ranking metrics comparing cross-encoder models across training epochs.

Model	Config	MAP	NDCG@10	MRR@10	P@10	R-Prec
GTE	AdamW-1	0.5005	0.7224	0.5940	0.8116	0.4964
	AdamW-2	0.4999	0.7203	0.5942	0.8256	0.5018
	AdamW-3	0.4899	0.6902	0.5972	0.8093	0.5017
	Lion-1	0.4794	0.6970	0.5854	0.7884	0.4814
	Lion-2	0.4577	0.6792	0.5957	0.7721	0.4642
	Lion-3	0.4521	0.6571	0.5931	0.7488	0.4662
MiniLM	AdamW-1	0.4814	0.7008	0.5828	0.8000	0.4884
	AdamW-2	0.4891	0.7094	0.5818	0.8116	0.4916
	AdamW-3	0.4908	0.7127	0.5826	0.8116	0.4943
	Lion-1	0.4858	0.7031	0.5890	0.8140	0.4952
	Lion-2	0.4755	0.6916	0.5942	0.8070	0.4803
	Lion-3	0.4706	0.6808	0.5988	0.8070	0.4809
ModernBERT	AdamW-1	0.5066	0.7105	0.5866	0.8163	0.5161
	AdamW-2	0.4893	0.6839	0.5886	0.7814	0.4946
	AdamW-3	0.4971	0.6959	0.5916	0.7860	0.5116
	Lion-1	0.5121	0.7142	0.5834	0.8163	0.5148
	Lion-2	0.5115	0.7225	0.5908	0.8209	0.5183
	Lion-3	0.5020	0.7051	0.5988	0.8256	0.5102

A.2 Precision and Recall Analysis

Tables A.2 and A.3 provide detailed precision and recall values at different cutoff thresholds, revealing the models' behavior at various result list depths.

TABLE A.2: Precision metrics at different cutoff levels.

Model	Config	P@5	P@10	P@20	P@100
GTE	AdamW-1	0.8930	0.8116	0.7372	0.3965
	AdamW-2	0.8698	0.8256	0.7326	0.3963
	AdamW-3	0.8558	0.8093	0.7128	0.3942
	Lion-1	0.8558	0.7884	0.7070	0.3893
	Lion-2	0.8326	0.7721	0.6802	0.3740
	Lion-3	0.8140	0.7488	0.6907	0.3740
MiniLM	AdamW-1	0.8698	0.8000	0.7151	0.3870
	AdamW-2	0.8698	0.8116	0.7186	0.3895
	AdamW-3	0.8558	0.8116	0.7256	0.3891
	Lion-1	0.8651	0.8140	0.7233	0.3847
	Lion-2	0.8744	0.8070	0.7081	0.3821
	Lion-3	0.8465	0.8070	0.6930	0.3809
ModernBERT	AdamW-1	0.8651	0.8163	0.7349	0.3993
	AdamW-2	0.8605	0.7814	0.6953	0.3912
	AdamW-3	0.8465	0.7860	0.6988	0.3958
	Lion-1	0.8791	0.8163	0.7314	0.4033
	Lion-2	0.8698	0.8209	0.7279	0.4016
	Lion-3	0.8651	0.8256	0.7093	0.3967

TABLE A.3: Recall metrics at different cutoff levels.

Model	Config	Recall@5	Recall@10	Recall@20	Recall@100
GTE	AdamW-1	0.1051	0.1689	0.2815	0.5501
	AdamW-2	0.1041	0.1760	0.2753	0.5538
	AdamW-3	0.1051	0.1715	0.2692	0.5489
	Lion-1	0.1002	0.1614	0.2684	0.5401
	Lion-2	0.0973	0.1610	0.2545	0.5211
	Lion-3	0.0929	0.1542	0.2574	0.5181
MiniLM	AdamW-1	0.1027	0.1652	0.2645	0.5426
	AdamW-2	0.1013	0.1668	0.2676	0.5446
	AdamW-3	0.1007	0.1702	0.2719	0.5427
	Lion-1	0.1022	0.1688	0.2704	0.5338
	Lion-2	0.1025	0.1641	0.2613	0.5338
	Lion-3	0.1001	0.1676	0.2542	0.5249
ModernBERT	AdamW-1	0.0995	0.1678	0.2731	0.5520
	AdamW-2	0.0979	0.1634	0.2603	0.5443
	AdamW-3	0.1019	0.1623	0.2627	0.5476
	Lion-1	0.1052	0.1689	0.2733	0.5542
	Lion-2	0.1048	0.1732	0.2699	0.5608
	Lion-3	0.1037	0.1722	0.2640	0.5570

A.3 NDCG Analysis

Table A.4 provides normalized discounted cumulative gain values at multiple cutoff points.

TABLE A.4: NDCG metrics at different cutoff levels.

Model	Config	NDCG@5	NDCG@10	NDCG@20	NDCG@100
GTE	AdamW-1	0.7567	0.7224	0.7071	0.6591
	AdamW-2	0.7343	0.7203	0.6967	0.6550
	AdamW-3	0.7080	0.6902	0.6703	0.6426
	Lion-1	0.7229	0.6970	0.6786	0.6433
	Lion-2	0.7008	0.6792	0.6580	0.6248
	Lion-3	0.6801	0.6571	0.6450	0.6124
MiniLM	AdamW-1	0.7207	0.7008	0.6833	0.6402
	AdamW-2	0.7345	0.7094	0.6908	0.6471
	AdamW-3	0.7244	0.7127	0.6949	0.6474
	Lion-1	0.7163	0.7031	0.6829	0.6369
	Lion-2	0.7099	0.6916	0.6706	0.6295
	Lion-3	0.6837	0.6808	0.6584	0.6268
ModernBERT	AdamW-1	0.7171	0.7105	0.6936	0.6555
	AdamW-2	0.7112	0.6839	0.6701	0.6431
	AdamW-3	0.7156	0.6959	0.6729	0.6497
	Lion-1	0.7285	0.7142	0.6991	0.6610
	Lion-2	0.7381	0.7225	0.6978	0.6638
	Lion-3	0.7165	0.7051	0.6797	0.6542

A.4 Metric Definitions

Table A.5 provides formal definitions of all evaluation metrics used in our experimental analysis.

TABLE A.5: Definitions of key evaluation metrics used in this thesis.

Metric	Definition
MAP	Mean Average Precision. The mean of average precision scores across all queries, where average precision is the average of precision values calculated at every position where a relevant document is retrieved. Range: [0,1].
NDCG@k	Normalized Discounted Cumulative Gain at rank k. Measures ranking quality with graded relevance and position-based discount. Range: [0,1].
P@k	Precision at k. The proportion of retrieved documents in the top k results that are relevant. Range: [0,1].
Recall@k	Recall at k. The proportion of all relevant documents that are retrieved in the top k results. Range: [0,1].
MRR@10	Mean Reciprocal Rank at 10. Average of the reciprocal ranks of the first relevant document within the top 10 results. Range: [0,1].
R-Prec	R-Precision. Precision after R documents retrieved, where R is the number of relevant documents for the query. Range: [0,1].
bpref	Binary Preference. Measures ranking quality when relevance judgments are incomplete by considering known relevant/non-relevant document pairs. Range: [0,1].

References

- [1] Payal Bajaj et al. “MS MARCO: A Human Generated MACHine Reading COMpre-hension Dataset.” In: *Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches @ NIPS 2016*. 2016. URL: <https://arxiv.org/abs/1611.09268>.
- [2] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020. URL: <https://www.wandb.com/>.
- [3] Xiangning Chen et al. *Symbolic Discovery of Optimization Algorithms*. 2023. arXiv: 2302.06675 [cs.LG]. URL: <https://arxiv.org/abs/2302.06675>.
- [4] Nick Craswell et al. *Overview of the TREC 2019 deep learning track*. 2020. arXiv: 2003.07820 [cs.IR]. URL: <https://arxiv.org/abs/2003.07820>.
- [5] Nick Craswell et al. *Overview of the TREC 2020 deep learning track*. 2021. arXiv: 2102.07662 [cs.IR]. URL: <https://arxiv.org/abs/2102.07662>.
- [6] Tri Dao et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. 2022. arXiv: 2205.14135 [cs.LG]. URL: <https://arxiv.org/abs/2205.14135>.
- [7] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2019, pp. 4171–4186. URL: <https://www.aclweb.org/anthology/N19-1423>.
- [8] John C. Duchi, Elad Hazan, and Yoram Singer. “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization.” In: *Journal of Machine Learning Research* 12.61 (Feb. 2011), pp. 2121–2159. URL: <https://www.jmlr.org/papers/volume12/duchi11a/duchi11a.pdf>.
- [9] Luyu Gao et al. *Complementing Lexical Retrieval with Semantic Residual Embedding*. 2021. arXiv: 2004.13969 [cs.IR]. URL: <https://arxiv.org/abs/2004.13969>.
- [10] Sebastian Hofstätter et al. “Efficiently teaching an effective dense retriever with balanced topic aware sampling.” In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2021, pp. 113–122.
- [11] Sebastian Hofstätter et al. *Improving Efficient Neural Ranking Models with Cross-Architecture Knowledge Distillation*. 2021. arXiv: 2010.02666 [cs.IR]. URL: <https://arxiv.org/abs/2010.02666>.

- [12] Minghao Hu et al. *Retrieve, Read, Rerank: Towards End-to-End Multi-Document Reading Comprehension*. 2019. arXiv: 1906.04618 [cs.CL]. URL: <https://arxiv.org/abs/1906.04618>.
- [13] Vladimir Karpukhin et al. “Dense passage retrieval for open-domain question answering.” In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 6769–6781.
- [14] Omar Khattab and Matei Zaharia. *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*. 2020. arXiv: 2004.12832 [cs.IR]. URL: <https://arxiv.org/abs/2004.12832>.
- [15] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [16] Zehan Li et al. *Towards General Text Embeddings with Multi-stage Contrastive Learning*. 2023. arXiv: 2308.03281 [cs.CL]. URL: <https://arxiv.org/abs/2308.03281>.
- [17] Jimmy Lin, Rodrigo Nogueira, and Andrew Yates. “Pretrained Transformers for Text Ranking: BERT and Beyond.” In: *Synthesis Lectures on Human Language Technologies*. 2021. URL: <https://arxiv.org/abs/2010.06467>.
- [18] Jimmy Lin et al. “Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations.” In: *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2021, pp. 2356–2360. URL: <https://dl.acm.org/doi/10.1145/3404835.3463238>.
- [19] Sheng-Chieh Lin, Jheng-Hong Yang, and Jimmy Lin. *Distilling Dense Representations for Ranking using Tightly-Coupled Teachers*. 2020. arXiv: 2010.11386 [cs.IR]. URL: <https://arxiv.org/abs/2010.11386>.
- [20] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. 2019. arXiv: 1711.05101 [cs.LG]. URL: <https://arxiv.org/abs/1711.05101>.
- [21] Modal. *Modal: Serverless compute for AI and data workflows*. <https://modal.com/>. Accessed: 20-04-2025. 2025.
- [22] National Institute of Standards and Technology (NIST). *trec_eval Information Retrieval Evaluation Software*. https://github.com/usnistgov/trec_eval. 2024.
- [23] Tri Nguyen et al. “MS MARCO: A Human Generated MACHine Reading COMprehension Dataset.” In: *CoRR* abs/1611.09268 (2016). arXiv: 1611.09268. URL: <http://arxiv.org/abs/1611.09268>.
- [24] Rodrigo Nogueira and Kyunghyun Cho. *Passage Re-ranking with BERT*. 2020. arXiv: 1901.04085 [cs.IR]. URL: <https://arxiv.org/abs/1901.04085>.

- [25] Rodrigo Nogueira and Jimmy Lin. “Passage Re-ranking with BERT.” In: *arXiv preprint arXiv:1901.04085* (2019).
- [26] Rodrigo Nogueira et al. “Document Ranking with a Pretrained Sequence-to-Sequence Model.” In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Online: Association for Computational Linguistics, Nov. 2020, pp. 1124–1136. URL: <https://aclanthology.org/2020.findings-emnlp.102>.
- [27] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: [1912.01703](https://arxiv.org/abs/1912.01703) [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.
- [28] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Nov. 2019. URL: <https://arxiv.org/abs/1908.10084>.
- [29] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Nov. 2019, pp. 3982–3992. URL: <https://arxiv.org/abs/1908.10084>.
- [30] Stephen Robertson and Hugo Zaragoza. “The probabilistic relevance framework: BM25 and beyond.” In: *Foundations and TrendsSM in Information Retrieval* 3.4 (2009), pp. 333–389.
- [31] Noam Shazeer. *GLU Variants Improve Transformer*. 2020. arXiv: [2002.05202](https://arxiv.org/abs/2002.05202) [cs.LG]. URL: <https://arxiv.org/abs/2002.05202>.
- [32] Jianlin Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: [2104.09864](https://arxiv.org/abs/2104.09864) [cs.CL]. URL: <https://arxiv.org/abs/2104.09864>.
- [33] Cole Thienes and Jack Pertschuk. *NBoost: Neural Boosting Search Results*. <https://github.com/koursaros-ai/nboost>. 2019.
- [34] Tijmen Tieleman and Geoffrey Hinton. *Lecture 6.5—RMSPProp: Divide the Gradient by a Running Average of Its Recent Magnitude*. Coursera: Neural Networks for Machine Learning. Accessed: Apr. 22, 2025. 2012. URL: https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
- [35] Wenhui Wang et al. *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*. 2020. arXiv: [2002.10957](https://arxiv.org/abs/2002.10957) [cs.CL]. URL: <https://arxiv.org/abs/2002.10957>.
- [36] Benjamin Warner et al. *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference*. 2024. arXiv: [2412.13663](https://arxiv.org/abs/2412.13663) [cs.CL]. URL: <https://arxiv.org/abs/2412.13663>.

- [37] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing.” In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. URL: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- [38] Lee Xiong et al. *Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval*. 2020. arXiv: 2007.00808 [cs.IR]. URL: <https://arxiv.org/abs/2007.00808>.
- [39] Guang Yu et al. “Improving dense retrieval with hard negatives from document reduction distillation.” In: *Findings of the Association for Computational Linguistics: EMNLP 2022*. 2022, pp. 3642–3652.
- [40] Michael Zhan et al. “Scaling Dense Retrievers to Billions of Passages.” In: *arXiv preprint arXiv:2502.05364* (2024).